2. DESCRIEREA STRUCTURII SISTEMELOR NUMERICE

OBIECTIVE

Capitolul 2 este dedicat prezentării principalelor modalităţi de descriere a sistemelor numerice. Este prezentat limbajul AHPL (A Hardware Programming Language), care va fi utilizat pe tot parcursul cursului pentru descrierea arhitecturii calculatorului didactic. Se exemplifică modalităţile de descriere a schemelor logice combinaţionale sub formă de Unităţi Logice Combinaţionale şi a schemelor logice secvenţiale sub formă de Module Logice.

CUPRINS

2. DESCRIEREA STRUCTURII SISTEMELOR NUMERICE 1

2.1 Modalităţi de descriere a structurii sistemelor numerice de prelucrare a datelor 2

2.2 Limbajul de descriere a structurilor numerice AHPL 3

2.3 Descrierea sistemelor numerice în AHPL 3
2.3.1 Tipuri de operanzi în AHPL 5
2.3.2 Tipuri de operatori 6
2.3.2.1 Operatori AHPL 6
2.3.3 Instrucţiuni AHPL 9
2.3.3.1 Instrucţiuni de atribuire 9
2.3.3.2 Instrucţiuni de salt 17

2.4 Descrierea în AHPL a schemelor logice combinaţionale 18
2.4.1 Sumatorul elementar complet 19
2.4.2 Sumator paralel pe 16 biţi 21
2.4.3 Unitate logică combinaţională pentru decodificator 23
2.4.4 Operaţiile de citire / scriere cu memoria în AHPL 23
2.4.4.1 Operaţia de scriere în memorie 24
2.4.4.2 Operaţia de citire din memorie 24
2.4.4.3 Unitate logică combinaţională pentru citire din memorie BUSFN 25

2.1 Modalităţi de descriere a structurii sistemelor numerice de prelucrare a datelor

Descrierea structurii sistemelor de prelucrare a datelor se poate realiza:

- fie examinând resursele hardware şi software ca entităţi diferite;
- fie ca entităţi echivalente.

Descrierea se poate efectua la diferite grade de detaliere, care pot fi organizate sub forma ierarhică.

În cele ce urmează structurile numerice vor fi reprezentate sub formă ierarhică pe mai multe nivele, componentele hardware şi software fiind tratate ca entităţi diferite.

În cadrul fiecărui nivel, sistemul numeric este caracterizat, din punct de vedere structural, printr-o mulţime finită de componente (unităţi funcţionale) care posedă anumite proprietăţi (atribute) şi o mulţime de modalităţi de interconectare a acestor elemente.

Fiecare element de la un anumit nivel de descriere îi corespunde pe nivelul imediat inferior un ansamblu de componente interconectate într-un mod specific.

Fiecare nivel de descriere se caracterizează printr-un limbaj distinct, specific componentelor şi modurilor de interconectare.

Nivelurile de descriere a structurilor numerice privite sub formă ierarhică sunt:

Nivelul elementelor de circuit;

Nivelul proiectării logice:

- subnivelul circuitelor de comutaţie (ASM);
- subnivelul transferurilor între registre (AHPL);

Nivelul interconectării resurselor (PMS);

Nivelul programelor:

- subnivelul setului de instrucţiuni (ISP);
- subnivelul limbajelor;
- subsistemul sistemelor de operare;
- subsistemul aplicaţiilor;

Nivelul elementelor de circuit - corespunde nivelului inferior de descriere. La acest nivel se operează cu:

- componente: rezistenţe, condensatori, tranzistori, etc.
- structura: scheme realizate cu aceste componente care au fost reprezentate grafic sau descrise analitic prin ecuaţii pe bucle şi noduri.

Acest nivel nu este specific sistemelor de calcul, fiind utilizat în special în studiul circuitelor electrice.

Nivelul proiectării logice - corespunde descrierii la nivelul circuitelor logice. În cadrul acestui nivel distingem :

a) - nivelul circuitelor de comutaţie - care se referă la circuitele combinaţionale şi la cele secvenţiale:

- componente: circuite logice elementare : SI, SAU, NU, DCD, MULTIPLEXOARE, BISTABILI, REGISTRE, etc.
- structura: scheme logice sau sisteme de ecuaţii logice, diagrame Karnaugh sau ASM (limbaj de descriere a maşinilor algoritmice de stare)

b) - subnivelul transferurilor între registre - care descrie comportarea sistemului la nivelul unităţii de execuţie şi a unităţii de comandă.

În cadrul descrierii unităţii de execuţie se operează cu:

- componente : unităţi logice combinaţionale, registre, memorii;
- structura : interconectarea componentelor prin instrucţiuni de conexiune specifice

În cadrul descrierii unităţii de comandă se operează cu:

- componente: algoritmi de comandă, microsecvenţiator;
- structura: descrierea paşilor de comandă prin instrucţiuni de transfer specifice, microinstrucţiuni şi microprogram.

Nivelul interconectarii resurselor - corespunde descrierii sistemelor de calcul la nivelul subansamblelor. Se operează cu:

- componente : P,M,S,L,P I/E,K,D,T.
- structura : calculatoare numerice, sisteme multiprocesor, reţele de calculatoare.

Acest nivel scoate în evidenţă interconectarea componentelor şi atributele intrinseci ale acestora.

Nivelul programelor - corespunde descrierii resurselor software ale sistemelor de calcul

Distingem patru subniveluri:

- subnivelul setului de instrucţiuni - operează cu:
  - componente : formatul instrucţiunii, moduri de adresare, registre;
  - structura : descrie modul logic de interpretare a setului de instrucţiuni. Se utilizează limbajul de descriere ISP.
- subnivelul limbajelor : descrie componentele limbajelor de programare;
- subnivelul sistemelor de operare: descrie resursele hardware şi sotfware;
- subnivelul aplicaţiilor: descrie caracteristicile programelor de aplicaţie pentru diverse domenii de activitate.

2.2 Limbajul de descriere a structurilor numerice AHPL

K. Iverson a propus limbajul APL (A Programming Language) care îndeplineste în mare măsură cerinţele descrierii arhitecturii sistemelor numerice şi într-o oarecare măsură, a unităţii de execuţie. Pornind de la un subset APL şi completîndu-l cu o serie de facilităţi specifice reprezentării operaţiilor la nivelul unităţii de comandă a unui subansamblu numeric, s-a dezvoltat un limbaj de descriere denumit AHPL (A Hardware Programming Language). Operaţiile reprezentate de AHPL au un echivalent direct în implementarea fizică. Totuşi, în scopul unei descrieri mai clare, se utilizează şi câteva convenţii din APL care nu au echivalent direct la nivelul resurselor fizice, fiind incluse şi acestea în AHPL.

2.3 Descrierea sistemelor numerice în AHPL

În AHPL un sistem numeric este descris cu ajutorul procedurilor şi funcţiilor.

Procedurile descriu module ce conţin elemente de memorie şi secvenţe de control.

Funcţiile descriu unităţi ce conţin elemente combinaţionale.

Structura unei proceduri este următoarea :

MODULE: < nume modul >

< declaraţii >
< secvenţa de control >

END SEQUENCE

< conexiuni >
< transferuri >

END

unde :

< nume modul >	- specifică numele modulului ce urmează să fie descris în AHPL
< declaraţiile >	- specifică elementele modulului la nivel de memorie, intrări, ieşiri, conexiuni cu exteriorul.
< secvenţa de control >	- specifică acţiunile ce se desfăşoară la diverse momente de timp, asociate paşilor AHPL.
END SEQUENCE	- specifică terminarea secvenţei de control.
< conexiuni>	
< transferuri >	- specifică conexiuni şi transferuri ce nu sunt asociate unui moment de timp (ele au loc tot timpul).
END	- specifică sfârşitul descrierii modulului.

Structura unei funcţii este următoarea:

UNIT: <nume funcţie> (lista de parametri)
<declaraţii>
<conexiuni>
END

Declaraţiile din cadrul modulului ajută la specificarea elementelor din care este constituit acesta. Limbajul AHPL recunoaste următoarele declaraţii:

MEMORY : <lista de variabile>
- specifică prin lista de variabile asociată toate elementele de memorie (bistabili, registre, memorii) utilizate în modulul respectiv.

INPUTS : <lista de variabile>
- specifică prin lista de variabile asociată, toate semnalele de intrare în modulul respectiv.

OUTPUTS : <lista de variabile>
- specifică prin lista de variabile asociată, semnalele de ieşire din modul.

BUSES : <lista de variabile >

- specifică prin lista de variabile asociată, magistralele din cadrul modulului.

COMBUSES: <lista de variabile>
- specifică prin lista de variabile asociată, magistralele (bidirectionale) de comunicaţie cu exteriorul.

ONE SHOTS:<lista de variabile> specifică prin lista de variabile asociată, toate elementele de tip monostabil din cadrul modulului.

În cadrul procedurilor ce descriu un modul pot fi utilizate ieşirile unităţilor logice combinaţionale specifi-cate prin funcţii.

2.3.1 Tipuri de operanzi în AHPL

Operanzii reprezintă variabile ce pot lua diverse valori numerice sau alfanumerice şi sunt asociaţi unor operatori ce aparţin limbajului.

Variabilele reprezintă nume prin care se pot face referiri simbolice la operanzi sau alte entităţi ale limbajului.

Operanzii cu care operează AHPL sunt de trei tipuri:
- **Scalari**, reprezentaţi prin litere mici: x,y,z;
- **Vectori**, reprezentaţi prin litere mari: X,Y,Z;
- **Matrice**, reprezentate prin litere mari aldine: **X,Y,Z**;

Prin **scalari** se reprezintă variabile independente, fără să li se asocieze o structură oarecare.
Exemplu : bistabili individuali din unitatea de comandă sau execuţie.

Vectorul este un ansamblu de operanzi scalari structuraţi sub forma unui tablou unidirectional. Numărul de componente ale unui vector formează dimensiunea acestuia şi poate fi determinata cu operatorul dimensiune ρ<nume_vector >.

Registrele unui calculator sunt reprezentate prin vectori.
De exemplu un registru RA de 16 biti are dimensiunea rRA=16, iar componentele sale se notează:

$RA = (RA_0, RA_1,...,RA_{\rho RA-1})$,

unde prin RA_i se specifică bitul din poziţia i a registrului RA.

Matricea este un ansamblu de operanzi structuraţi într-un tablou bidimensional. Matricea are deci două dimensiuni şi în mod corespunzător există doi operanzi dimensiune ρ1 şi ρ2 .

ρ2**M** =numărul de coloane ;
ρ1**M** =numărul de linii.

Astfel o matrice **M** de dimensiunea (ρ1**M**, ρ2**M**) se prezintă astfel:

$$M = \begin{bmatrix} M_0^0 & M_1^0 & M_{\rho 2M-1}^0 \\ M_0^1 & M_1^1 & M_{\rho 2M-1}^1 \\ \cdots & \cdots & \cdots \\ M_0^{\rho 1M-1} & M_1^{\rho 1M-1} & M_{\rho 2M-1}^{\rho 1M-1} \end{bmatrix}$$

Vectorul $\left(M_0^i \quad M_1^i \quad \cdots \quad M_{\rho 2M-1}^i\right)$ reprezintă linia i, iar

Vectorul $\left(M_j^0 \quad M_j^1 \quad \cdots \quad M_j^{\rho 1M-1}\right)$ reprezintă coloana j a matricei **M**.

Prin matrice se reprezintă ansamble de registre grupate pentru diferite operaţii sau module de memorie. Astfel un modul de memorie de 4 K cuvinte de 16 biţi poate fi reprezentat ca o matrice **M** cu ρ1**M**=4096 şi ρ2**M**=16.

M^i reprezintă cuvântul i din memorie,

iar M_j^i reprezintă bitul j din cuvântul i.

Selecţia unui subvector sau submatrici se face foarte simplu, în mod explicit astfel:

$A_{i:j}$ reprezintă vectorul format cu elementele de la i până la j ale vectorului A.

$\mathbf{M}^{i:j}$ reprezintă matricea formata cu liniile de la i până la j ale matricii **M**, şi reprezintă în general o zonă de memorie.

2.3.2 Tipuri de operatori

Prin operatori se specifică prelucrările ce se aplică asupra operanzilor. În funcţie de numărul operanzilor, operatorii pot fi unari sau binari iar în funcţie de tipul operanzilor, operatorii pot fi primitivi, dacă se aplică asupra unor variabile scalare sau operatori micşti, dacă se aplică asupra unor combinaţii de variabile de tip scalar, vectorial sau matriceal.

2.3.2.1 Operatori AHPL

Operatorii de care dispune limbajul AHPL sunt:

- operatori logici:

$\wedge$, $\vee$, $\bar{}$ sau . , + , $\bar{}$ care realizează funcţiile logice SI, SAU, NU între operanzii specificaţi.

Exemple:

Fie: $A = (A_0, A_1, ... , A_n)$
$B = (B_0, B_1, ... , B_n)$
doi vectori şi a , b doi scalari
$a \wedge b$;
$A + B = (A_0+B_0, A_1+B_1, ..., A_n+B_n)$

- operatori de reducere:
$\wedge/$; $\vee/$; +/ realizează funcţia logică specificată între componentele vectorului asupra căruia acţionează. Rezultatul este un scalar.
Exemple:

$\wedge/A = A_0 \wedge A_1 \wedge A_2 \wedge ... \wedge A_n$

- operatori de sincronizare
SYN <nume_semnal_asincron>
SL <nume_semnal>
Operatorul SYN sincronizează semnalul specificat ca parametru (nume_semnal_asincron) , cu semnalul de ceas al sistemului.
Operatorul SL generează un semnal de o durată egală cu perioada semnalului de ceas indiferent de durata semnalului specificat ca parametru (nume_semnal).
Exemplu:
SL(SYN(starta)) generează un semnal sincron cu semnalul de ceas şi de o durată egală cu perioada semnalului de ceas, indiferent de durata semnalului **starta**.

- operatori de selecţie:
j ; i:j selectează biţi dintr-un registru sau zone de memorie.
Exemple:
A_j - selectează bitul j din registrul A
$A_{i:j} =(A_i, A_{i+1}, ..., A_j)$ - selectează grupul de biţi ai registrului A între poziţiile i şi j
$\mathbf{M}^{i:j}$ - selectează zona de memorie cuprinsă între adresele i şi j

-operator de selecţie prin comprimare
X/Y elimină componentele din Y corespunzătoare componentelor egale cu 0 din X
Exemplu:
Fie $A = (0, 1, 1, 0, 1, 0, 0, 1)$
$B = (B_0, B_1, B_2, B_3, B_4, B_5, B_6, B_7)$
$A / B = (B_1, B_2, B_4, B_7)$

- operator de selecţie prin comprimare şi reducere:
M*F realizează eliminarea liniilor din matricea M corespunzătoare componentelor egale cu 0 din vectorul F
Exemplu:
Fie

$$M = \begin{bmatrix} M^0 \\ M^1 \\ M^2 \end{bmatrix}$$ şi F=[0,0,1]

rezultă:

M*F selectează linia 2, adică $\mathbf{M}^2$

-operatori de concatenare

, concatenare pe linii

! concatenare pe coloane

Exemplu:

Fie a şi b doi scalari, A şi B doi vectori iar **M** o matrice

A=a,b reprezintă un vector care are ca şi componente pe a şi b

M=A!B reprezintă o matrice care are ca linii componetele A şi B

De notat faptul că dacă vectorii concatenaţi pe coloane nu au aceeaţi dimensiune se face aliniere la dreapta.

- operator de codificare

n T m; realizează codificarea binară a valorii reprezentată de numărul m într-un vector care are n componente binare. Rezultatul codificării este un vector.

Exermplu:

4 T 5 generează vectorul (0,1,0,1)

8 T 7 generează vectorul (0,0,0,0,0,1,1,1)

- operatori de decodificare

⊥

DCD; realizează funcţia inversă a codificării adică interpretează componentele vectorului căruia i se aplică ca cifre binare şi generează o valoare numerică. În general această valoare reprezintă o adresă.

Exemplu:

⊥ (0,1,0,1) generează valoarea 5

DCD (0,0,0,0,0,1,0,1) generează valoarea 5

- operator de atribuire

a) de conexiune = ; realizează conectarea resurselor specificate

b) de transfer ← ; realizează transferul informaţiei de la sursă la destinaţie. Resursa destinaţie trebuie să fie element cu posibilitate de memorare.

Exemplu:

a = b conectează semnalul b la semnalul a

Z = A conectează registrul A la ieşirile Z ale unui modul

B←A conţinutul registrului A se transferă în registrul B

- operator de dimensionare

ρ; aplicat unui vector, generează numărul de comonente ale acestuia (lungimea vectorului). Fie A= (A_0, A_1, ..., A_n) atunci ρA generează valoarea **n+1**

- operatori relaţionali

x R y

unde: R ∈ { < ; ≤ ; = ; ≥ ; > ; ≠ } stabileşte relaţia între cei doi operanzi şi generează valoarea adevărat (1 logic) dacă relaţia este îndeplinită, şi generează valoarea fals (0 logic) în caz contrar. În general relaţia R este implementată cu ajutorul funcţiilor, prin unităţi logice combinaţionale .

2.3.3 Instrucţiuni AHPL

Secvenţa de control a unei unităţi de comandă se specifică cu ajutorul paşilor AHPL.

În continuare se vor prezenta atât modul în care se specifică diferite acţiuni în limbajul AHPL cât şi implementarea acestora la nivelul resurselor fizice.

Un **pas AHPL** specifică toate acţiunile ce au loc în perioada de timp cât secvenţa de control se găseşte în starea respectivă. Durata unui pas AHPL este egală cu perioada semnalului de ceas (de tact).

Un pas AHPL poate deci specifica mai multe instrucţiuni care au loc în acelaţi moment de timp. Instrucţiunile AHPL sunt separate prin caracterul (**;**).

Un pas AHPL poate fi prezentat în mai multe linii sursă ce specifică instrucţiuni sau comentarii. Fiecare pas AHPL specifică atât acţiunile ce au loc în pasul respectiv cât şi adresa următoarei instrucţiuni din secvenţa de control.

Pasul AHPL următor poate fi specificat în mod explicit prin instrucţiune de salt sau în mod implicit ca fiind următorul în textul programului.

Fiecare instrucţiune poate conţine comentarii formate din şiruri de caractere încadrate de delimitatori: /* comentariu */.

Instrucţiunie AHPL pot fi împărţite în două mari categorii:

- Instrucţiuni de atribuire;
- Instrucţiuni de salt.

Sfârşitul secvenţei de paşi AHPL este marcat cu declaraţia END SEQUENCE., iar sfârşitul unui program AHPL este marcat cu declaraţia END.

2.3.3.1 Instrucţiuni de atribuire

Formatul unei instrucţiuni de atribuire este:

<destinaţie> <operator_de_atribuire> <expresie>

Valoarea obţinută prin evaluarea expresiei este atribuită variabilei destinaţie. Dacă vectorii destinaţie şi sursă nu au aceeaţi dimensiune, se face alinierea dreapta.

Expresiile pot fi formate utilizând:

- operatori logici: ($\wedge$;$\vee$; $^-$);
- operatori de sincronizare: (SL ; SYN (semnal_asincron));
- operatori de selecţie: (A_j; $A_{m:n}$; $\mathbf{M}^j$; $\mathbf{M}^{m:n}$);
- operator de selecţie prin comprimare: (X / Y);
- operator de selecţie prin comprimare şi reducere: (**M** * F);
- operatori de concatenare: (A,B ; A!B);
- operator de codificare: (n T p);
- operatori de decodificare: ($\perp^X$; DCD);
- operatori relaţionali.

Pentru scrierea expresiilor trebuie să se ţină seama de următoarele:

- indicii superiori şi inferiori trebuie specificaţi prin valori constante (nu se admit expresii de indici);
- expresiile (sau subexpresiile) din paranteze se evaluează cu prioritate;
- pentru a evita proliferarea parantezelor se vor avea în vedere priorităţile operatorilor astfel:

1°. Negaţie şi SYN -prioritatea cea mai mare;
2°. Operatori de selectie;
3°. SI logic;
4°. SAU şi SAU EXCLUSIV;
5°. Concatenare

Ca operanzi pentru scrierea expresiilor se pot utiliza:

- intrări provenite din alte module;
- funcţii logice combinaţionale;
- elemente de memorie (vectori, matrice);
- magistrale de date sau comenzi;
- constante binare.

Instrucţiunile de atribuire sunt împărţite în două categorii în funcţie de operatorul care se foloseşte:

- **instrucţiuni de conexiune** (operatorul de conexiune =);
- **instrucţiuni de transfer** (operatorul de transfer $\leftarrow$).

Prin operaţiile de conexiune valoarea expresiei se atribuie variabilei doar pe durata pasului AHPL în care este specificată conexiunea respectivă. Deci acţiunea are un caracter temporar. Prin operaţiile de transfer, valoarea expresiei se atribuie variabilei din partea stânga, când secvenţa de control a ajuns în pasul AHPL respectiv, şi este memorată în variabila destinaţie care păstrează aceasta valoare până la o nouă atribuire. Acţiunea are un caracter permanent specific funcţionării circuitelor secvenţiale.

1°. Instrucţiuni de conexiune

Dacă se notează cu:

MAG - o magistrală predeclarată
Z - un vector de ieşire;
VLCO - un vector de funcţii logice combinaţionale de ieşire;
MLCO - o matrice de vectori logici combinaţionali VLCO;
F - un vector de constante binare sau funcţii logice

instrucţiunile de conexiune pot fi specificate prin una din următoarele forme:

Z = VLCO
MAG = VLCO
Z = **MLCO***F
MAG = **MLCO***F

În primele două instrucţiuni are loc o conexiune necondiţionată a ieşirilor VLCO la intrările Z sau MAG.

Fiecare componentă a vectorului VLCO poate fi o funcţie logică combinaţională a cărei ieşire va fi conectată la intrarea corespunzătoare a vectorului destinaţie.

Ultimele două instrucţiuni specifică o conexiune condiţionată în sensul că, prin F se specifică linia din matricea de vectori de funcţii logice **MLCO,** care se va conecta la Z sau MAG. Operaţia MLCO*F este echivalentă cu operaţiile de comprimare şi reducere.

Astfel dacă: F=(f,g,h) iar **MLCO** = (A!B!C),

operaţia MLCO*F este echivalentă cu

$(A \wedge f) \vee (B \wedge g) \vee (C \wedge h)$

ceea ce poate fi scris şi sub forma:

$(A_0 \wedge f, A_1 \wedge f, .., A_n \wedge f) \vee (B_0 \wedge g, B_1 \wedge g, .., B_n \wedge g) \vee (C_0 \wedge h, C_1 \wedge h, .., C_n \wedge h)$

În general astfel de expresii se utilizează pentru specificarea operaţiilor de multiplexare (când +/F=1), dar se poate ca şi (+/F > 1) să fie utilă în diferite situaţii.

Pentru exemplificare se considera pasul AHPL următor:

10.BUS=(A!B) * (f_1 ,f_2)

unde A=(A_0 ,A_1) şi B= (B_0, B_1)

Implementarea fizică a acestui pas este prezentată în Fig. 2.1. Se vede că operaţia de conexiune are loc numai dacă bistabilul 10 este în 1, deci controlul este la pasul AHPL 10.

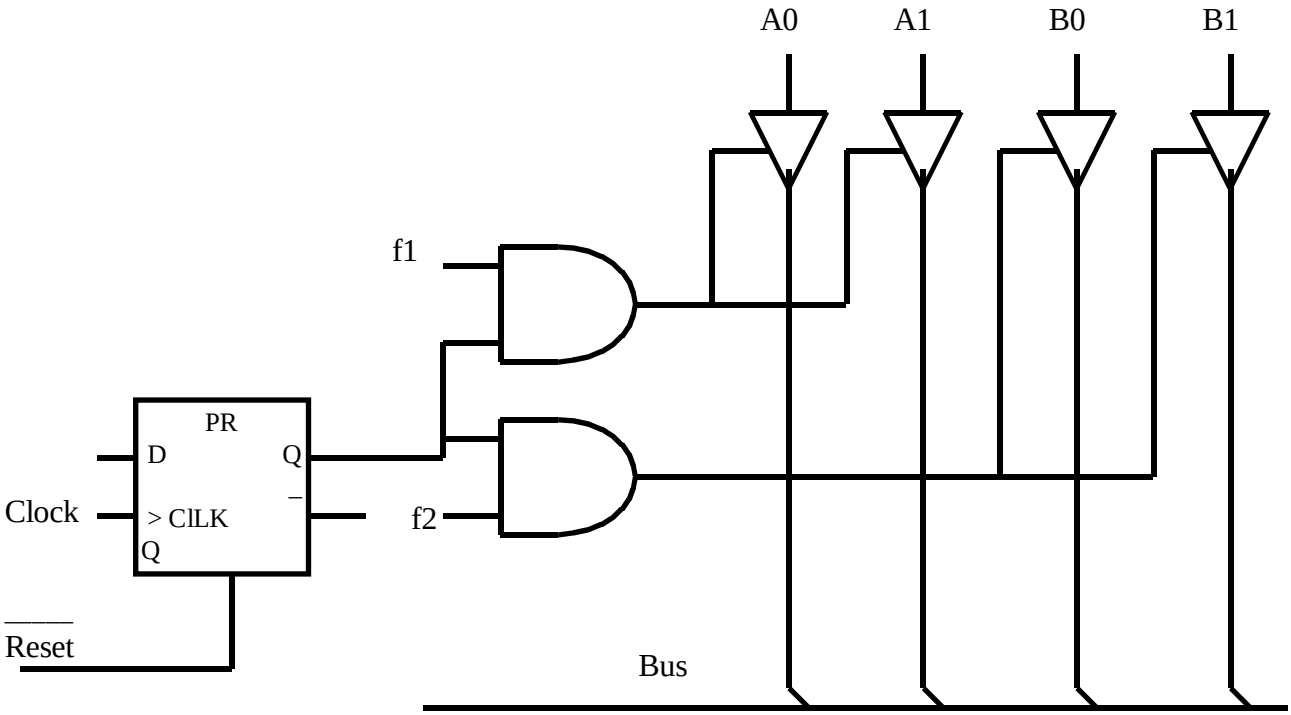


Fig.2.1 Conexiune selectiva

2°. Instrucţiuni de transfer

Variabila destinaţie este un bistabil sau un vector ce are ca echivalent un registru sau un cuvânt de memorie. Înscrierea valorii expresiei în operandul destinaţie se face sincronizat cu un semnal de tact. Acest semnal de tact poate fi specificat explicit printr-o conexiune sau implicit ca fiind tactul unităţii de comandă.

Forma generală a unei instrucţiuni de transfer este:

VD ←VLCO

VD ← **MLCO***F

MD*F←VLCO

unde: VD este un vector destinaţie iar

MD este o matrice destinaţie

Prima instrucţiune specifică un transfer neconditionat.

Cea de a doua formă poate fi descompusă în următoarele operaţii elementare:

- conexiune la intrările vectorului destinaţie, a vectorului rezultat prin evaluarea expresiei si
- apoi înscrierea acestuia în VD.

Cea de a treia formă este echivalentă cu: valoarea lui VLCO se transferă în toate liniile matricei rezultate prin comprimarea şi reducerea cu vectorul de selectie F.

Exemplu:

Fie

MLCO = (A!B!C)

F = (f,g,h)
VD = D

Instrucţiunea:

D←(A!B!C)*(f,g,h) se va implementa astfel:

D←(A ∧ f) ∨ (B ∧ g) ∨ (C ∧ h).

Instrucţiunea:

(A!B!C)*(f,g,h) ←D se va implementa astfel:

(A!B!C) ←(((D!A)*(f,))!((D!B)*(g,))!((D!C)*(h,)))

sau

dacă f=1 A←D altfel A rămâne neschimbat
dacă g=1 B←D altfel B rămâne neschimbat
dacă h=1 C←D altfel C rămâne neschimbat

Diferenţa dintre cele două implementări este următoarea:

În primul caz **transferul se execută întotdeauna** datele de intrare fiind selectate în mod corespunzător. În al doilea caz funcţiile de transfer condiţionează intrarea de tact şi deci transferul are loc numai dacă se îndeplineşte condiţia.

Exemplu:

10. A*f←B se va implementa astfel:

Instrucţiunea de mai sus se va implementa ca în Fig 2.2.a. când transferul este condiţionat pe intrări, şi ca în Fig. 2.2.b, când transferul este condiţionat pe tact.

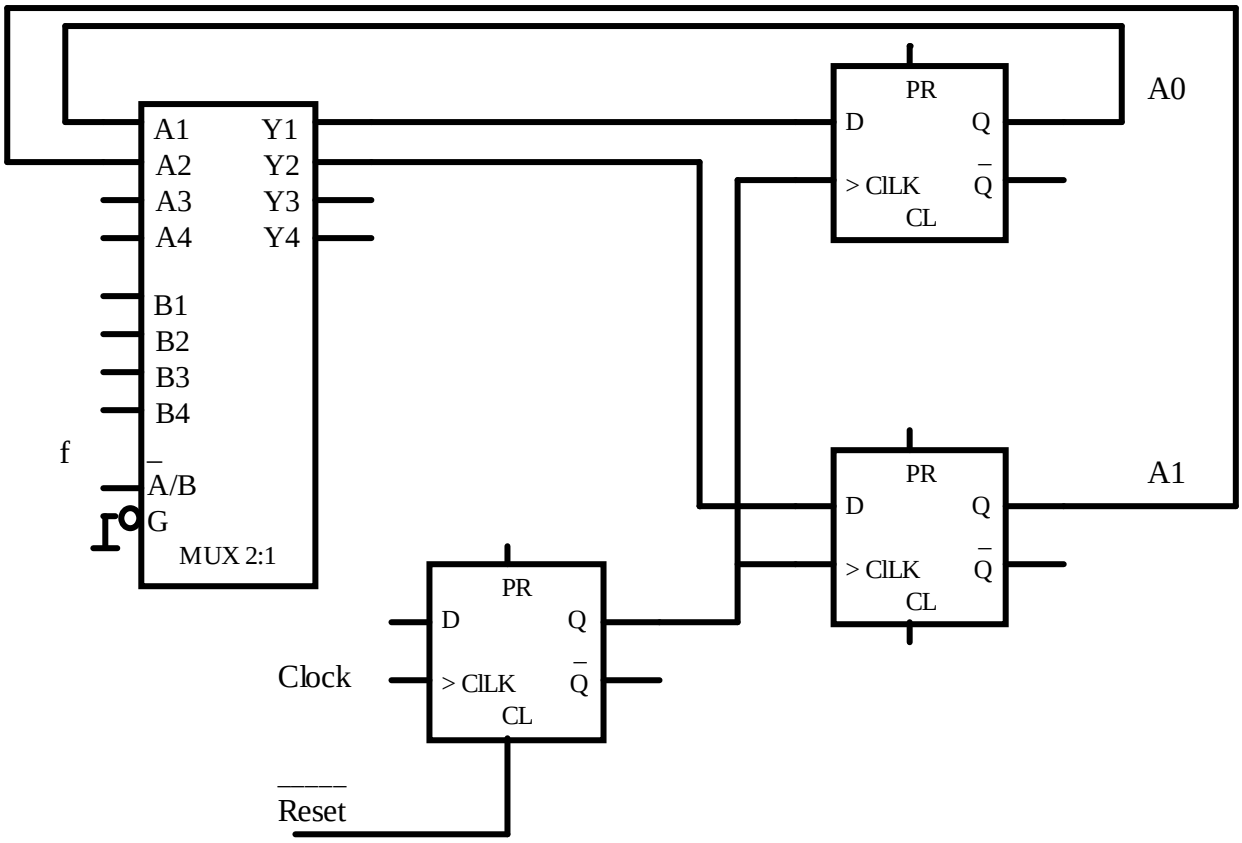


Fig.2.2a Transfer conditionat pe intrari

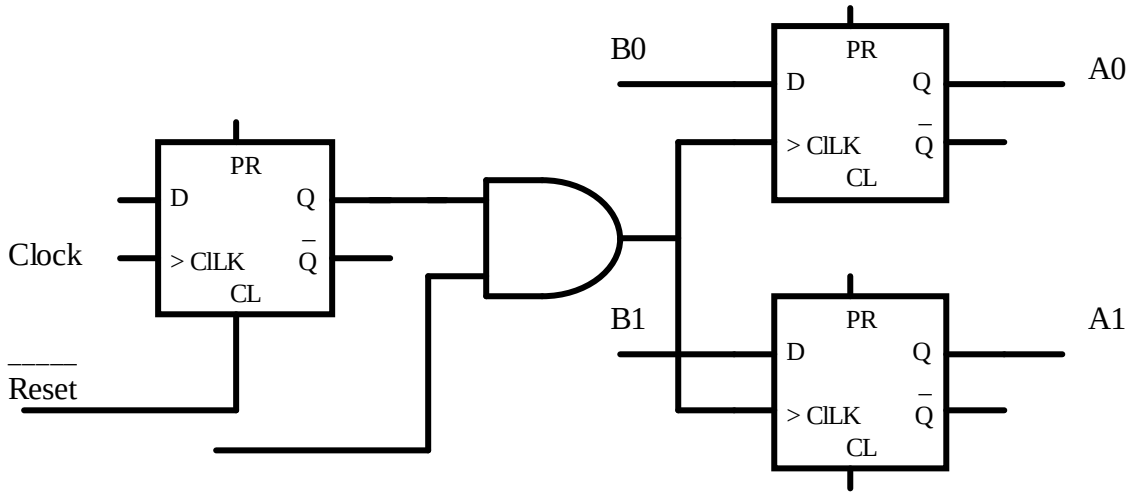


Fig.2.2b Transfer conditionat pe tact

3°. Specificarea semnalului de tact

Se observă că în ambele implementări tactul propriu-zis este dat de trecerea în 1 a bistabilului 10 care marchează execuţia programului la acest pas (trecere care este sincronă cu semnalul se tact al modulului).

Uneori este necesar ca transferul (condiţionat sau necondiţionat) să se execute sincronizat cu unul din fronturile semnalului de tact (clock) sau cu un tact extern ceea ce permite transferuri asincrone în raport cu semnalul de ceas al modulului.

Acesta se poate realiza prin specificarea, la pasul respectiv a unei conexiuni de tipul:

CK = semnal_de_tact

când toate resursele din pasul AHPL respectiv vor fi comandate de un semnal de tact extern, sau

CK [nume_resursa] = semnal_de_tact

pentru a evidenţia că numai intrările de tact ale resursei specificate sunt conectate la "semnal_de_tact".

Astfel considerînd că toţi bistabilii şi registrele comută pe front crescător se consideră următoarele exemple:

```
15. A←B      /* transferul are loc pe frontul crescător al semnalului de tact al modulului
                (CLOCK) */

16. CK[A] = TACT;
        A←B  /* transferul are loc pe frontul crescător al unui tact extern (TACT) */

17. CK[A] =    ; CK[C]=TACT
        A←B; /* transferul are loc pe frontul crescător al semnalului  */
        C←B; /* transferul are loc pe frontul crescător  al semnalului  TACT */
        D←B; /* transferul are loc pe frontul crescător al semnalului  CLOCK */

18. CK = ;
        A←B;18
        C←B;
        D←B; /* toate transferurile, la acest pas, sunt efectuate  pe frontul crescător al lui
                CLOCK*/
```

În toate aceste cazuri intrarea de tact va fi condiţionată de valoarea expresiei din instrucţiunea CK = expresie

Astfel instrucţiunea:

```
15. CK[A] = ;
        A←B
```

Se va implementa ca în Fig. 2.3.

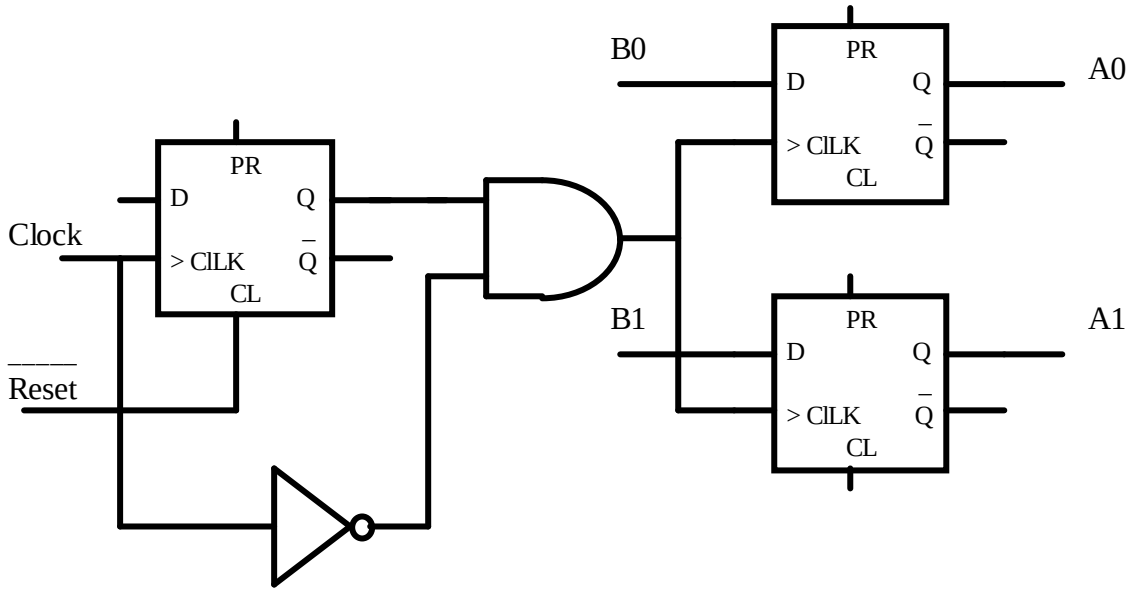


Fig.2.3. Transfer pe frontul căzător al tactului

4°. Transferuri şi conexiuni după END SEQUENCE

Transferurile şi conexiunile specificate în secvenţa de control sunt acţiuni care au loc la pasul respectiv, când execuţia secvenţei de comandă ajunge în acel punct.

Există situaţii în care este necesară o conexiune cu caracter permanent, nu numai pe o perioadă a semnalului de tact. De asemenea, în anumite situaţii este necesar ca un transfer să se execute la fiecare perioadă de tact. Astfel de transferuri şi conexiuni se specifică după declaraţia END SEQUENCE şi bineânţeles, înainte de END.

Exemplu:

Conexiunea Z = (A!B)*(f_1 ,f_2)

unde A=(A_0 ,A_1) şi B=(B_0 ,B_1) specificată după END SEQVENCE se va implementa ca în Fig. 2.4. Se observă că nu apare condiţionarea cu vreun pas AHPL, conexiunea are loc tot timpul indiferent de punctul în care se găseşte execuţia secvenţei de control.

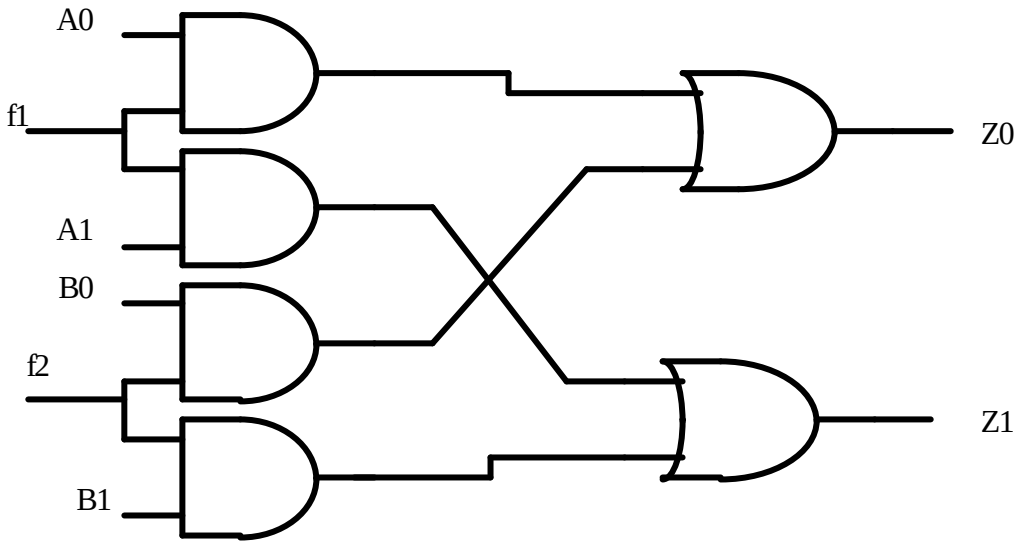


Fig. 2.4. Conexiune dupa END SEQUENCE

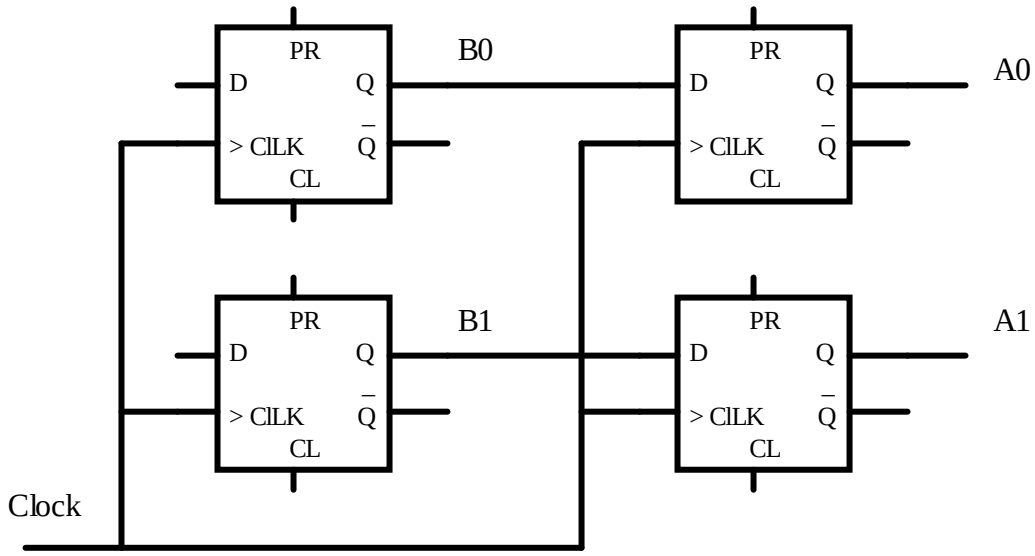


Fig. 2.4. Transfer dupa END SEQUENCE

Transferul A←B specificat după END SEQUENCE se va implementa ca în Fig. 2.5.
Se vede că registrul A "urmăreşte" registrul B.

Dacă A şi B sunt în module diferite, cu semnele de tact diferite, se realizează un transfer sincron cu tactul modulului destinaţie.

2.3.3.2 Instrucţiuni de salt

Cu ajutorul instrucţiunilor de salt se pot implementa structuri de control de tip: ramificaţii în program, cicluri, secvenţe paralele.

Formatul instrucţiunilor de salt este:

1°. $\rightarrow (S_i)$	Salt necondiţionat
2°. $\rightarrow$ (F)/(S)	Salt condiţionat, unde $\rho F = \rho S$
3°. $\rightarrow$ (S)	Salt multiplu (necondiţionat), $\rho S > 1$

unde:

S_i este un număr reprezentând pasul AHPL cu care se va continua secvenţa de control, şi trebuie specificat printr-o constantă.

F este un vector de funcţii logice reprezentând condiţiile de test.

S este un vector de constante reprezentând numerele de paşi AHPL la care se poate continua secvenţa, funcţie de valoarea vectorului F.

Pentru saltul condiţionat se evaluează expresiile pentru F şi prin comprimarea lui S cu F, se obţin numerele paşilor AHPL cu care se continua execuţia.

Există următoarele cazuri:

$\rho((F)/(S)) = 0$	Se continuă cu pasul următor.
$\rho((F)/(S)) = 1$	Se execută ca şi saltul necondiţionat.
$\rho((F)/(S)) > 1$	Se iniţiază o secvenţă paralelă începând cu paşii AHPL selectaţi.

Saltul prezentat la cazul 3 reprezintă un salt multiplu necondiţionat, ce initiază deci o secvenţă paralelă.

În secvenţele de comandă care nu au o structură ciclică sau în cazul secvenţelor paralele se poate specifica terminarea execuţiei unei secvenţe prin instrucţiunea:

DEAD END

care va trece unitatea de comandă într-o stare terminală. În această stare nu se mai execută nici o acţiune dintre cele specificate în secvenţa de comandă până la o nouă iniţializare. DEAD END poate apare oriunde în secvenţa de comandă.

Pentru a facilita scrierea programelor în care au loc mai multe acţiuni în aceaşi perioadă de tact, limbajul AHPL permite specificarea execuţiei mai multor paşi AHPL în aceeaţi perioadă de timp. Astfel dacă la un pas AHPL se specific declaraţia:

NO DELAY

toate acţiunile din acest pas se desfăşoară în perioada de tact corespunzătoare pasului precedent din care provine (ultimul care nu conţine declaraţia NO DELAY).

2.4 Descrierea în AHPL a schemelor logice combinaţionale

Dacă fiecare pas AHPL dintr-o secvenţă de comandă conţine declaraţia NO DELAY, iar în cazul secvenţei se specifică numai conexiuni se obţine o schemă logică combinaţională. O astfel de secvenţă se poate scrie şi după END SEQUENCE.

Pentru a facilita scrierea secvenţelor de descriere a circuitelor logice combinaţionale oferind şi o flexibilitate de utilizare a acestora în diferite module, limbajul AHPL permite descrierea separată ca module de program distincte sub forma:

UNIT: nume_funcţie <lista de parametri>
<declaraţii>
<conexiuni>
END

Dacă o schemă combinaţională apare (este apelată) de mai multe ori cu aceeaşi parametri efectivi ea va fi generată fizic o singură dată cu condiţia să nu se depăşească fan-out-ul ieşirilor.

Dacă însă este apelată de mai multe ori, cu parametri diferiţi se poate proceda în două moduri:

- se implementează câte o copie a schemei pentru fiecare apelare;
- se implementează schema o singură dată dar se prevede schema de selectie a parametrilor de intrare corespunzători fiecărei apelări.

Alegerea uneia dintre modalităţi se face prin analiza complexităţii funcţiei combinaţionale şi a logicii de selecţie a parametrilor.

Apelul unei unităţi logice combinaţionale se face prin:

nume (lista_de_parametri_efectivi).

Deci în expresii AHPL pot să apară nume de unităţi logice combinaţionale definite.

În continuare se vor prezenta câteva unităţi logice combinaţionale foarte utilizate în structurile numerice.

2.4.1 Sumatorul elementar complet

Sumatorul elementar complet realizează operaţia de adunare a unor operanzi de lungime un bit, ţinând seama de transportul de intrare şi generează pe lângă sumă şi eventualul transport.

În Fig.2.6 se prezintă schema bloc a unui sumator elementar complet. Intrările sunt cei doi operanzi şi transportul din exterior iar ieşirile sunt suma şi transportul spre exterior.

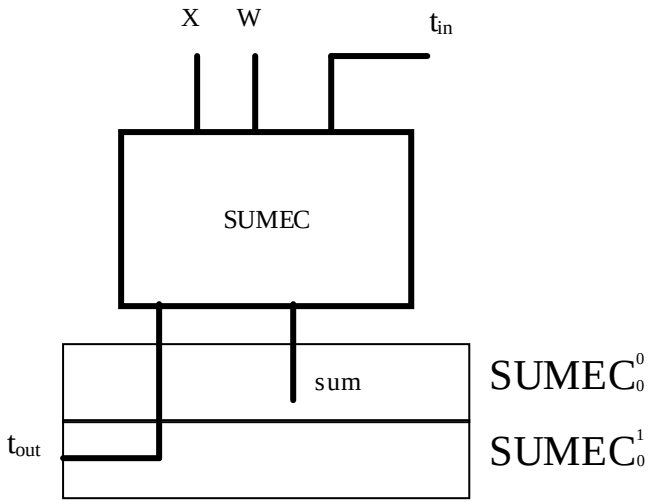


Fig. 2.6. Sumator elementar complet

Tabela de adevăr, diagramele Karnaugh şi ecuaţiile logice sunt prezentate în continuare. Ecuaţiile logice rezultate sunt :

$sum = x \oplus y \oplus t_{in}$

$t_{out} = ((x \oplus y) \wedge t_{in}) \vee (x \wedge y)$

Tabela de adevăr:

x	y	t_{in}	suma	t_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Aceste expresii logice pot fi descrise în AHPL astfel:

UNIT: SUMEC $(x;y;t_{in})$
INPUTS: $(x;y;t_{in})$
OUTPUTS:SUMEC [2;1]
1. $a = x \oplus y$
2. $b = x \wedge y$
3. $sum = a \oplus t_{in}$
4. $c = a \oplus t_{in}$
5. $t_{out} = b \vee c$
6. SUMEC = sum
7. SUMEC = t_{out}
END

Implementarea unităţii logice combinaţionale a sumatorului elementar complet este prezentată în Fig. 2.7.

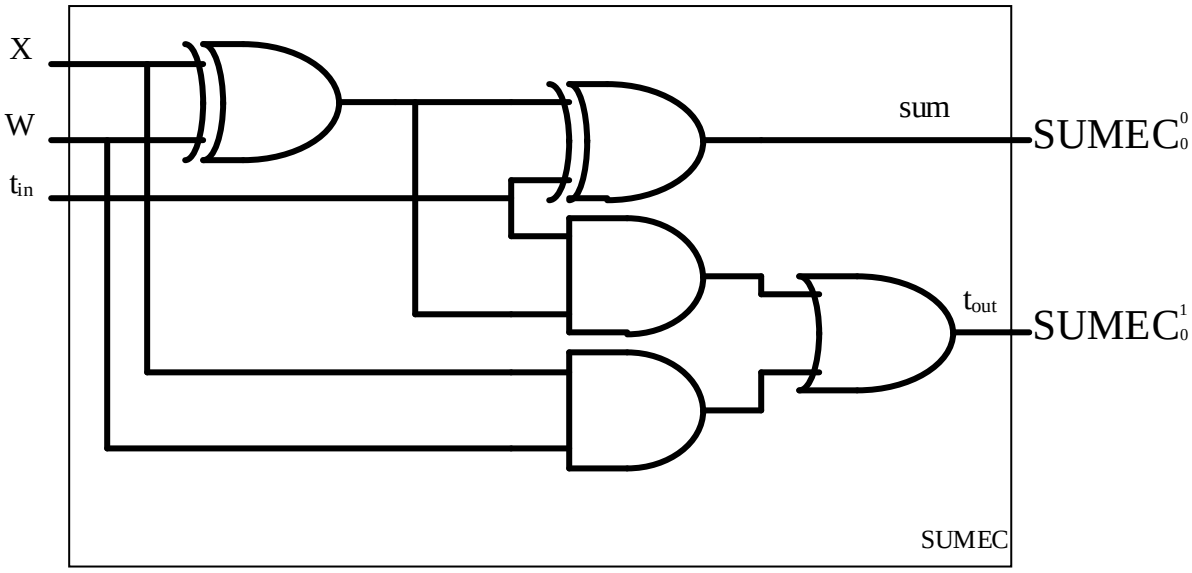


Fig. 2.7. Schema sumator elementar complet

2.4.2 Sumator paralel pe 16 biţi

Pentru a descrie un sumator paralel pe 16 biţi se va considera un vector de funcţii SUMEC obţinându-se astfel o matrice cu 2 linii reprezentând suma şi transportul pe fiecare rang.
Operanzii sumatorului paralel sunt:

$X = (X_0, X_1 ,..., X_{15})$
$Y = (Y_0, Y_1 ,..., Y_{15})$
Suma şi transportul din rangul cel mai semnificativ vor fi:
$ADD=(ADD_0, ADD_1 ,...,ADD_{16})$

Descrierea AHPL a sumatorului pe 16 biţi este:

UNIT: ADD (X; Y; t_{16})
INPUTS: X[16]; Y[16];t_{16}
OUTPUTS: ADD [17]
1. T = $SUMEC^1_{0:15}$ (X; Y; $T_{1:15}$,t_{16})
2. S = $SUMEC^0_{0:15}$ (X; Y; $T_{1:15}$,t_{16})
3. ADD = T_0 ,$S_{0:15}$
END

Implementarea sumatorului paralel este dată în Fig. 2.8.

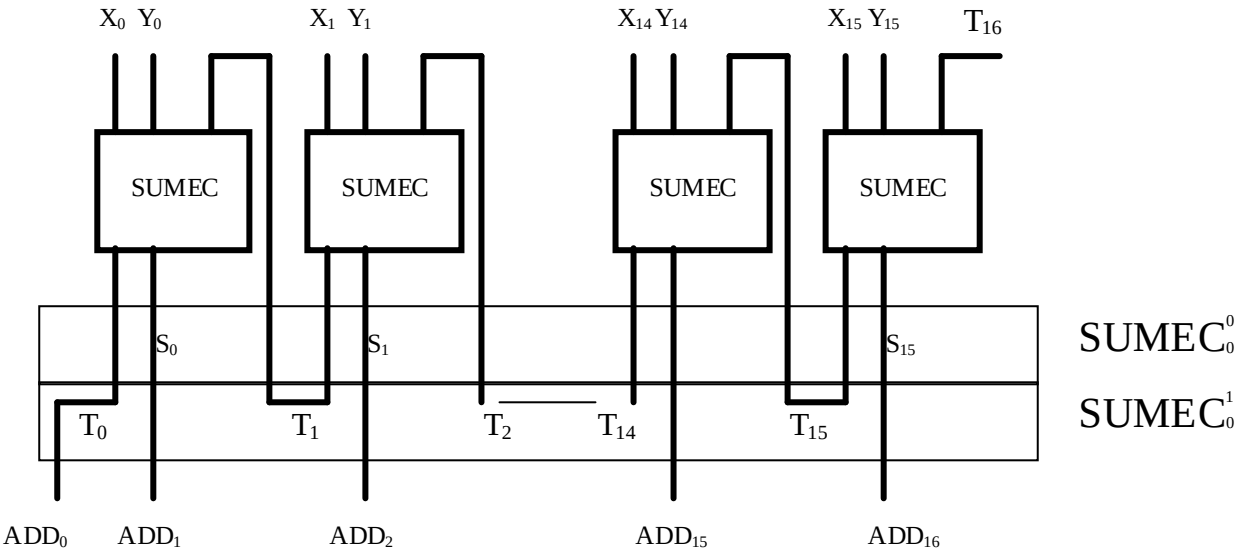


Fig. 2.8. Sumator paralel pe 16 biţi

Pentru efectuarea unor operaţii aritmetice ca: adunare, scădere, complementare, sumatorul paralel proiectat se poate utiliza într-o schemă ca şi cea din Fig 2.9.

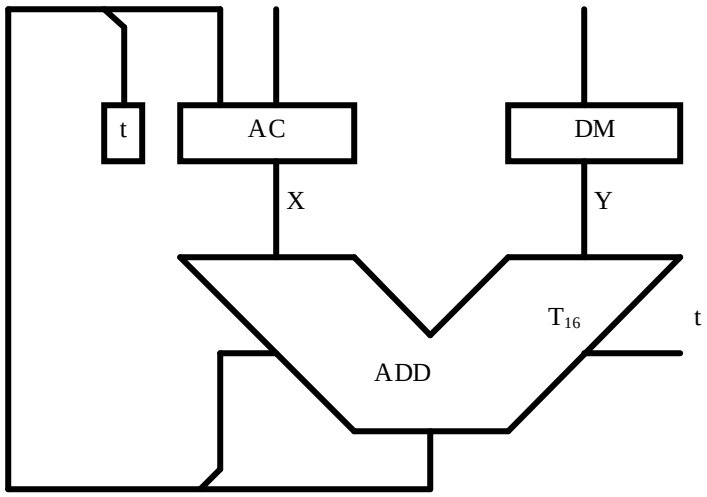


Fig. 2.9. Utilizarea sumatorului paralel

Cu schema din Fig.2.9 se pot efectua următoarele operaţii:

t,AC←ADD (AC ; DM ; 0) /* AC←AC + DM */

t,AC←ADD (AC ; $\overline{DM}$; 1) /* AC←AC - DM */

t,AC←ADD (AC; DM ; 1) /* AC←DM - AC */

2.4.3 Unitate logică combinaţională pentru decodificator

Un decodificator primeşte la intrare un vector binar oarecare de n biţi şi generează la ieşire un vector de 2^n biţi astfel că un singur bit are valoarea 1, ceilalţi având valoarea 0.

Schema bloc a unui decodificator de la n la 2^n este arătată în Fig. 2.10.

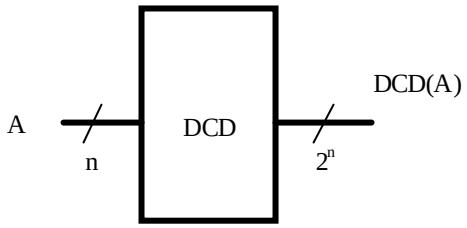


Fig. 2.10. Decodificator cu n intrări şi 2^n ieşiri

Bitul i din cei 2^n biţi de ieşire va fi:

$$DCD_i = \begin{cases} 0 \text{ daca } \perp A \neq i \\ 1 \text{ daca } \perp A = i \end{cases}$$

Unitatea ce descrie un decodificator paralel va fi descrisă astfel:

```
UNIT: DCD (A)
        INPUTS:A[n]
        OUTPUTS:DCD[m]      /* unde m = 2^n */
        1.   A = (~A0, ~A1, ~A2, ....., ~An-1)
        2.   DEi = ∧/((nTi)/A), ((~nTi)/~A))
        3.   DCD = DE0:m-1
END
```

2.4.4 Operaţiile de citire / scriere cu memoria în AHPL

Se consideră o memorie **M** [2^n ,m] şi un vector de selecţie F cu $\rho F = 2^n$ componente astfel ca +/F=1 (o singură componentă este egală cu 1).

Presupunem că datele se citesc şi se scriu în memorie printr-un registru de date DM[m]. Schema generală pentru acces la memorie este prezentată în Fig. 2.11.

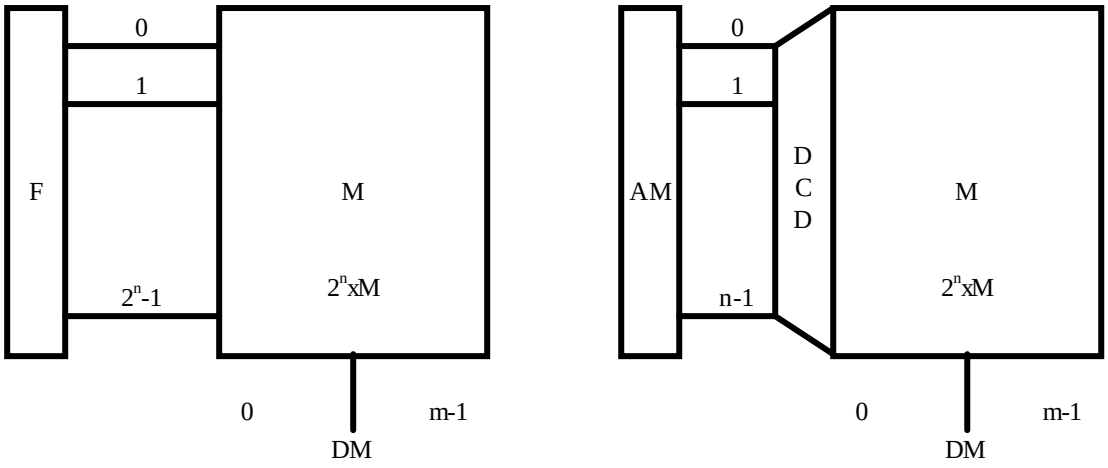


Fig. 2.11 Accesul la memorie

2.4.4.1 Operaţia de scriere în memorie

Scrierea datelor din DM în M se va face cu instrucţiunea:

M*F ← DM

În schema din Fig. 2.12 vectorul de selectie F a fost înlocuit cu o schemă logică combinaţională care primeşte la intrare un vector de n biţi, registrul de adrese AM, şi generează un vector decodificat de 2^n biţi care are o singură componentă egală cu 1 în orice moment. Această schemă logică este realizată de DCD (AM).

Instrucţiunea de scriere în memorie va fi deci:

M*DCD(AM) ←DM

2.4.4.2 Operaţia de citire din memorie

Pentru citire din memorie se va utiliza o funcţie specială denumită BUSFN, care are ca parametri memoria şi adresa de la care se doreşte citirea.

Formatul instrucţiunii de citire din memorie este:

DM←**BUSFN** ((M;DCD(AM))

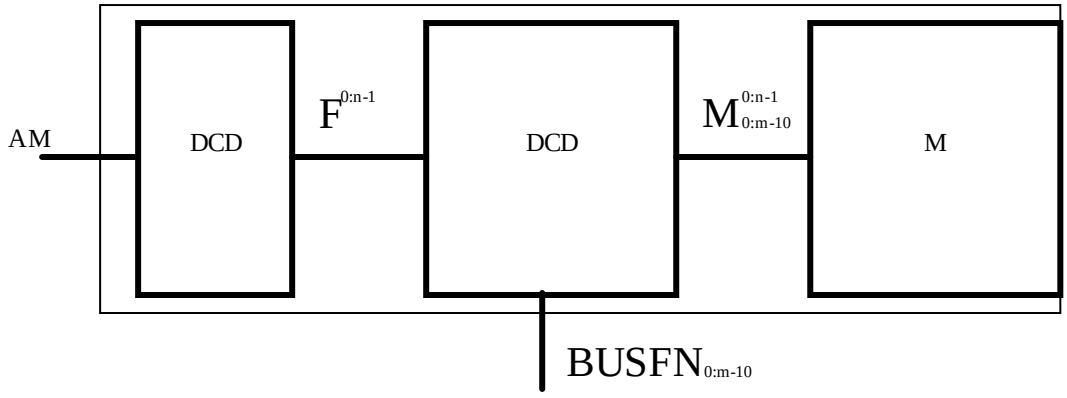


Fig.2.12 Citirea din memorie

2.4.4.3 Unitate logică combinaţională pentru citire din memorie BUSFN

Operaţia de citire din memorie constă în selectarea celulei de memorie a cărei adresa este specificată de liniile de adresă. Prin utilizarea unui vector F, asociat funcţiei logice de decodificare a adresei DCD (AM), se poate scrie o unitate logică combinaţională ce realizează selectarea celulei de memorie specificată.

Fie o memorie **M** ce are capacitatea ρ1**M** = n şi ρ2**M** = m, n cuvinte de memorie a m biţi fiecare, şi un vector F asociat decodificatorului de adresă : $F^{0:n-1}$ =DCD (AM) cu proprietatea că +/F=1.

Funcţia de citire din memorie BUSFN se poate descrie astfel:

UNIT: BUSFN (**M**;F)
INPUTS: **M** [n;m]; F[n]
OUTPUTS: BUSFN [m]
1.$\mathbf{NIVS}^{0:n-1} = \mathbf{M}^{0:n-1} \wedge F^{0:n-1}$ /* **NIVS**=**M**∧F */
2.$\mathbf{NIV}^{0:n-1} = \mathbf{NIVS}^{0} ! (\mathbf{NIVS}^{1:n-1} \vee \mathbf{NIV}^{0:n-2})$
3.BUSFN=$\mathbf{NIV}^{n-1}$
END

Exemplu:

Fie o memorie de 4 cuvinte a 8 biţi. Unitatea logică combinaţională ce asigură citirea din aceasta memorie are următoarea descriere:

UNIT BUSFN (**M**,F)
INPUTS: **M** [4,8], F [4]
OUTPUTS: BUSFN [8]
1. $\mathbf{NIVS} = \mathbf{M} \wedge F$
2. $\mathbf{NIV} = \mathbf{NIVS}^{0} ! (\mathbf{NIVS}^{1:3} \vee \mathbf{NIV}^{0:2})$
3. BUSFN=$\mathbf{NIV}^{3}$
END

Implementarea unităţii logice combinaţionale este prezentată în Fig 2.13.

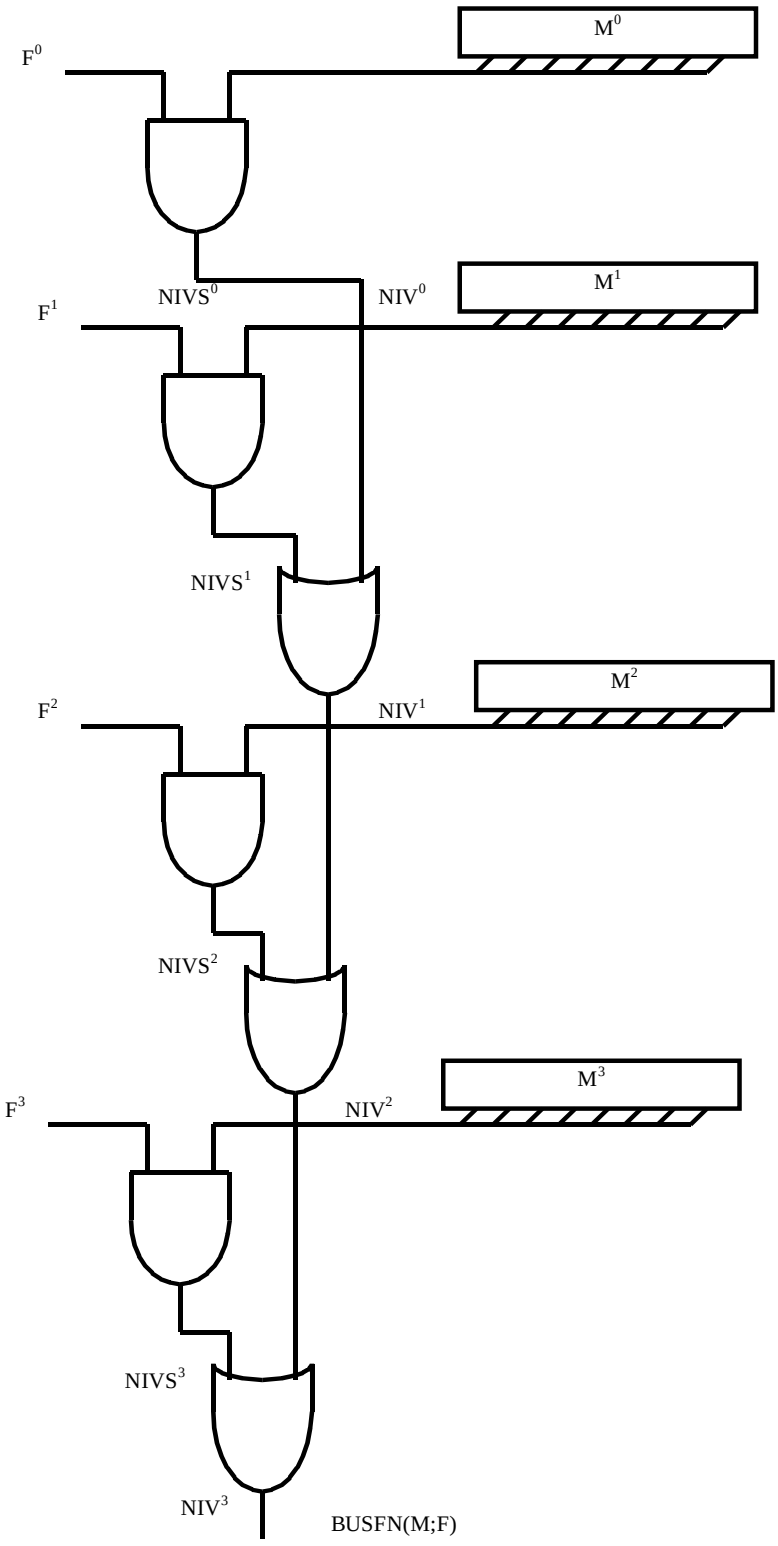


Fig. 2.13 Implementarea unității BUSFN